Name: Sanskar Soni
Subject: JAVA
Assignment: 8
Form Number: 251101107

//Problem 1: Division by Zero Task: Write a program that takes two integers from the user and
//performs division. Use a try block to perform the division, and a catch block to handle the ArithmeticException in case
//of division by zero. Ensure that a finally block prints a message indicating that the operation is complete.
Solution:

```
package ExceptionHendling;
import java.util.*;
public class DivisionbyZero {
public static void main(String[] args) {
	Scanner sc = new Scanner(System.in);
	System.out.println("Enter the First No");
	int a = sc.nextInt();
	System.out.println("Enter the second No");
	int b = sc.nextInt();
	try {
		int result = a/b;
		System.out.println("Result"+ result);
	}
	catch(ArithmeticException e){
	 System.err.println("cant be divided by 0");
	}
	finally {
		System.out.println("Program Sucewssfully Excuted");
	}
	System.out.println("Rest of the Code Follow ....");
}
}
```

```java
//Problem 1: Division by Zero Task: Write a program that takes two integers t
//performs division. Use a try block to perform the division, and a catch blo
//of division by zero. Ensure that a finally block prints a message indicatin
package ExceptionHendling;
import java.util.*;
public class DivisionbyZero {
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the First No");
    int a = sc.nextInt();
    System.out.println("Enter the second No");
    int b = sc.nextInt();
    try {
        int result = a/b;
        System.out.println("Result"+ result);
    }
    catch(ArithmeticException e){
     System.err.println("cant be divided by 0");
    }
    finally {
        System.out.println("Program Sucewssfully Excuted");
    }
    System.out.println("Rest of the Code Follow ....");
```

Problems @ Javadoc Declaration Console

```
<terminated> DivisionbyZero [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.exe (24-Mar-2026, 2:20:5
Enter the First No
45
Enter the second No
0
cant be divided by 0
Program Sucewssfully Excuted
Rest of the Code Follow ....
```

//Problem 2: Array Index Out of Bounds
//Task: Write a program that initializes an array of integers and tries to
//access an index that is out of bounds. Use a try block to access the array
//and a catch block to handle the ArrayIndexOutOfBoundsException.
//Ensure a finally block prints a message indicating the operation is complete
Solution:

```java
package ExceptionHendling;

public class ArrayIndexOutofBounds {
        public static void main(String[] args) {
                int arr[] = {1, 2, 3};

                try {
```

```
            System.out.println(arr[5]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Invalid Index");
            }finally {
                System.out.println("Done");
            }
        }
}
```

```
//Problem 2: Array Index Out of Bounds
//Task: Write a program that initializes an array of integers and tries
//access an index that is out of bounds. Use a try block to access the a
//and a catch block to handle the ArrayIndexOutOfBoundsException.
//Ensure a finally block prints a message indicating the operation is co

package ExceptionHendling;

public class ArrayIndexOutofBounds {
    public static void main(String[] args) {
        int arr[] = {1, 2, 3};

        try {
        System.out.println(arr[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid Index");
        }finally {
            System.out.println("Done");
        }
    }
}
```

Problems @ Javadoc Declaration Console

```
terminated> ArrayIndexOutofBounds [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\jav
Invalid Index
Done
```

//Problem 3: Null Pointer Exception
//Task: Write a program that initializes a string variable to null and
//then tries to call a method on it. Use a try block to call the method
//and a catch block to handle the NullPointerException. Ensure a finally
//block prints a message indicating the operation is complete

Solution:

package ExceptionHandling;

```java
package ExceptionHendling;
//Problem 3: Null Pointer Exception
//Task: Write a program that initializes a string variable to null and
//then tries to call a method on it. Use a try block to call the method
//and a catch block to handle the NullPointerException. Ensure a finally
//block prints a message indicating the operation is complete

public class NullTest {
public static void main(String[] args) {
    String str = null;

    try {
        System.out.println(str.length());
    } catch (NullPointerException e) {
        System.out.println("Object is null...");
    }
    finally {
        System.out.println("Finally will execute no matter what..");
    }
    System.out.println("Rest the code follow...");
    }
}
```

Problems @ Javadoc Declaration Console

```
<terminated> NullTest [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.exe (24-Mar-2026,
Object is null...
Finally will execute no matter what..
Rest the code follow...
```

```java
public class NullTest {
public static void main(String[] args) {
        String str = null;

        try {
                System.out.println(str.length());
        } catch (NullPointerException e) {
                System.out.println("Object is null...");
        }
        finally {
                System.out.println("Finally will execute no matter what..");
        }
        System.out.println("Rest the code follow...");
        }
```

```
}
```

//Problem 4: Number Format Exception
//Task: Write a program that takes a string input from the user and tries
//to convert it to an integer. Use a try block to perform the conversion and
//a catch block to handle the NumberFormatException. Ensure a finally block
//prints a message indicating the operation is complete.

Solution:

```
package ExceptionHendling;
import java.util.*;
public class Number_Format_Exception {
public static void main(String[] args) {
	Scanner Sc = new Scanner(System.in);
	System.out.println("Enter the String");
	try {
		String a = Sc.nextLine();
		int x = Integer.parseInt(a);
		System.out.println("The string is " + x);
	}catch(NumberFormatException e){
		System.err.println("This conversion not possible");
	}
	finally {
		System.out.println("Finally always Run");
	}
	System.out.println("Rest of the code follow ...");
}
```

Calculator.java | Mathtest.java | DivisionbyZero.java | ArrayIndexOutofBounds.java | NullTest.java | *Number_Format_Exception.java

```java
//Problem 4: Number Format Exception
//Task: Write a program that takes a string input from the user and tries
//to convert it to an integer. Use a try block to perform the conversion and
//a catch block to handle the NumberFormatException. Ensure a finally block
//prints a message indicating the operation is complete.
package ExceptionHendling;
import java.util.*;
public class Number_Format_Exception {
public static void main(String[] args) {
    Scanner Sc = new Scanner(System.in);
    System.out.println("Enter the String");
    try {
        String a = Sc.nextLine();
        int x = Integer.parseInt(a);
        System.out.println("The string is " + x);
    }catch(NumberFormatException e){
        System.err.println("This conversion not possible");
    }
    finally {
        System.out.println("Finally always Run");
    }
    System.out.println("Rest of the code follow ...");
}
}
```

Problems | Javadoc | Declaration | Console

```
<terminated> Number_Format_Exception [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.exe (24-Mar-
Enter the String
java
This conversion not possible
Finally always Run
Rest of the code follow ...
```

//Problem:5 Nested Try Blocks with Multiple Exceptions
//Task: Write a program that demonstrates the use of
//nested try blocks. The program should perform the following tasks:
//try should have two separate try blocks.
//In the first nested try:
//Divide two integers, handling any potential ArithmeticException.
//Within the second try block, initialize an array and attempt to access
//an out-of-bounds index, handling the ArrayIndexOutOfBoundsException.
//Ensure that appropriate messages are printed for each exception,
//and that a final message is printed indicating the completion of
//the operation.

Solution:

```
package ExceptionHendling;

public class NestedException {
	public static void main(String[] args) {
		try {
			try {
				int a = 5;
				int b= 0;
				System.out.println("The Dividion is " + (a /b));
			}catch(ArithmeticException e) {
				System.err.println("Not to be divided By ");
			}
			try {
				int [] arr = {3,5,7,8};
				System.out.println("The arry is "+ arr[5]);

			}catch(ArrayIndexOutOfBoundsException e) {
				System.err.println("Array index out of bounds");
			}
			}catch(Exception e) {
			System.err.println("Not allowed Here ");
		}
		finally {
			System.out.println("Finally always run");
		}
		System.out.println("rest of the Code follow ");
	}
	}
```

java - Java7_1Exception/src/ExceptionHendling/NestedException.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

DivisionbyZero.java | ArrayIndexOutofBounds.java | NullTest.java | Number_Format_Exception.java | *NestedException.java

```java
//Problem:5 Nested Try Blocks with Multiple Exceptions
//Task: Write a program that demonstrates the use of
//nested try blocks. The program should perform the following tasks:
//try should have two separate try blocks.
//In the first nested try:
//Divide two integers, handling any potential ArithmeticException.
//Within the second try block, initialize an array and attempt to access
//an out-of-bounds index, handling the ArrayIndexOutOfBoundsException.
//Ensure that appropriate messages are printed for each exception,
//and that a final message is printed indicating the completion of
//the operation.
package ExceptionHendling;

public class NestedException {
    public static void main(String[] args) {
        try {
            try {
                int a = 5;
                int b= 0;
                System.out.println("The Dividion is " + (a /b));
            }catch(ArithmeticException e) {
                System.err.println("Not to be divided By ");
            }
            try {
            }catch(ArrayIndexOutOfBoundsException e) {
                System.err.println("Array index out of bounds");
            }
            }catch(Exception e) {
            System.err.println("Not allowed Here ");
        }
        finally {
            System.out.println("Finally always run");
        }
        System.out.println("rest of the Code follow ");
    }
    }
```

Console

<terminated> NestedException [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.exe (24-Mar-2026,

```
Not to be divided By
Array index out of bounds
Finally always run
rest of the Code follow
```

Problem6 : Create Customer class with the relevant information like name, address, id, phone no etc. Write a parameterized constructor and relevant methods (disp(), etc) appropriately.
Create Account class with account type, account number, minimum balance etc., Write calculateInterest method (use simple interest – assume time and rate appropriately).
Create a user defined exception class "NegativeBalanceException" and throw that exception when there is negative balance while calculating the interest.
Use Account class in Customer class display the details of customer with account information. (No Inheritance – use association only)

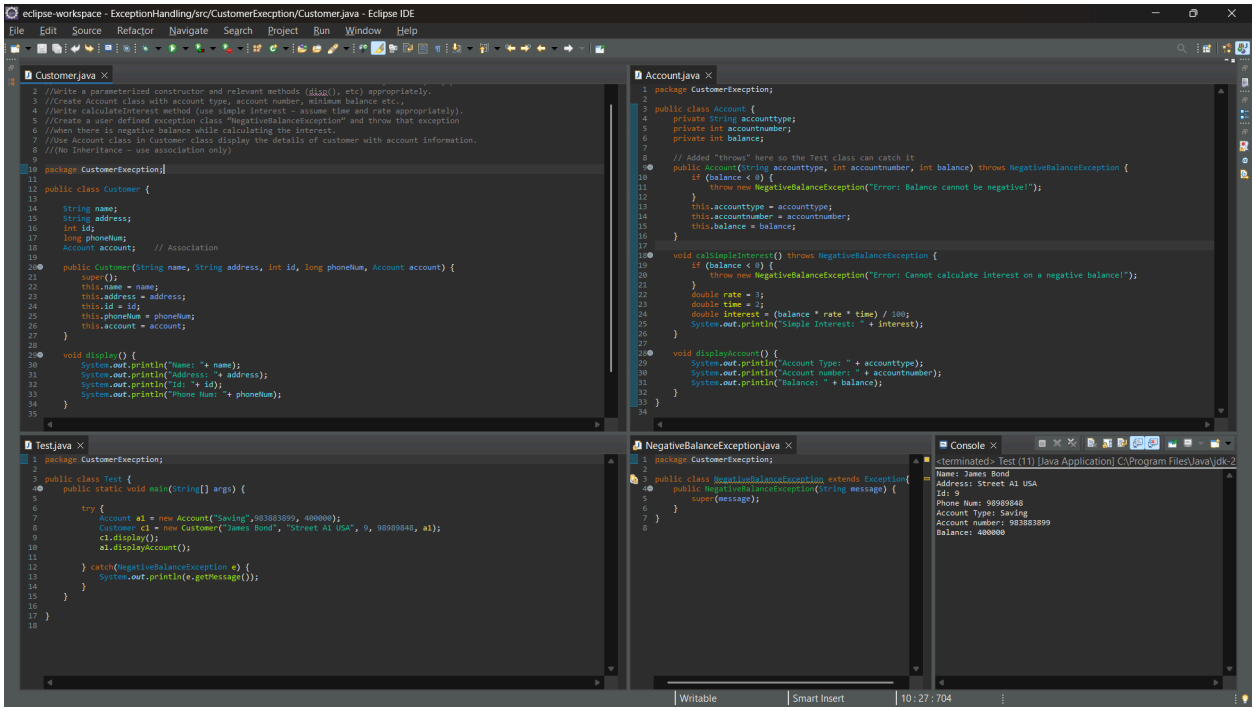
Solution:

```
package CustomerExecption;

public class Customer {

	String name;

	String address;

	int id;

	long phoneNum;

	Account account;	// Association


	public Customer(String name, String address, int id, long phoneNum, Account account) {

		super();

		this.name = name;

		this.address = address;

		this.id = id;

		this.phoneNum = phoneNum;

		this.account = account;

	}


	void display() {

		System.out.println("Name: "+ name);

		System.out.println("Address: "+ address);

		System.out.println("Id: "+ id);

		System.out.println("Phone Num: "+ phoneNum);

	}
```

```
}

package CustomerExecption;

public class Account {

   private String accounttype;

   private int accountnumber;

   private int balance;

   // Added "throws" here so the Test class can catch it

   public Account(String accounttype, int accountnumber, int balance) throws
NegativeBalanceException {

       if (balance < 0) {

           throw new NegativeBalanceException("Error: Balance cannot be negative!");

       }

       this.accounttype = accounttype;

       this.accountnumber = accountnumber;

       this.balance = balance;

   }

   void calSimpleInterest() throws NegativeBalanceException {

       if (balance < 0) {

           throw new NegativeBalanceException("Error: Cannot calculate interest on a negative
balance!");

       }

       double rate = 3;

       double time = 2;

       double interest = (balance * rate * time) / 100;

       System.out.println("Simple Interest: " + interest);
```

```
  }
  void displayAccount() {
      System.out.println("Account Type: " + accounttype);
      System.out.println("Account number: " + accountnumber);
      System.out.println("Balance: " + balance);
  }
}
```

```
package CustomerExecption;
public class NegativeBalanceException extends Exception{
        public NegativeBalanceException(String message) {
                super(message);
        }
}
```

```
package CustomerExecption;
public class Test {
        public static void main(String[] args) {

                try {
                        Account a1 = new Account("Saving",983883899, 400000);
                        Customer c1 = new Customer("James Bond", "Street A1 USA", 9,
98989848, a1);
                        c1.display();
                        a1.displayAccount();
```

```
            } catch(NegativeBalanceException e) {
                System.out.println(e.getMessage());
            }
        }
}
```